TRAFFICCARE
UNIT TESTING REPORT

Smart Traffic Complaint System

PROJECT UNIT TESTING DOCUMENTATION

1. Introduction

TrafficCare is a Smart Traffic Complaint System designed to allow users to report traffic-related problems by submitting complaints and uploading images.

The system uses a MERN-based architecture integrated with a deep-learning model to classify road conditions and identify issues such as road damage and potholes.

Unit testing is performed to verify that individual modules and components of the application function correctly and produce the expected results.

This report documents the unit testing performed on the backend, frontend and deep-learning modules of the TrafficCare project.

2. Objectives of Unit Testing

The main objectives of unit testing are:

1. To verify the correctness of individual software modules.
2. To identify errors in authentication and authorization functionality.
3. To validate complaint management operations.
4. To verify database connection handling.
5. To test image validation and preprocessing.
6. To verify the behavior of frontend React components.
7. To verify deep-learning prediction API responses.
8. To ensure that invalid inputs are handled appropriately.
9. To improve the reliability and maintainability of the application.

3. Testing Environment

The following technologies and tools were used during unit testing.

Component	Technology
Operating System	macOS
Backend	Node.js, Express.js
Frontend	React, Vite
Backend Testing Framework	Jest

Component	Technology
Frontend Testing Framework	Vitest
Deep Learning API	Python, FastAPI
DL Testing Framework	Python unittest
API Testing	FastAPI TestClient
Deep Learning Framework	PyTorch
Image Processing	Pillow
Database	MongoDB
Model Architecture	ResNet18

4. Backend Unit Testing

The backend handles user authentication, complaint management, administrative operations, middleware processing and database connectivity.

Backend unit testing was performed using Jest.

Test execution command:

```
npm test
```

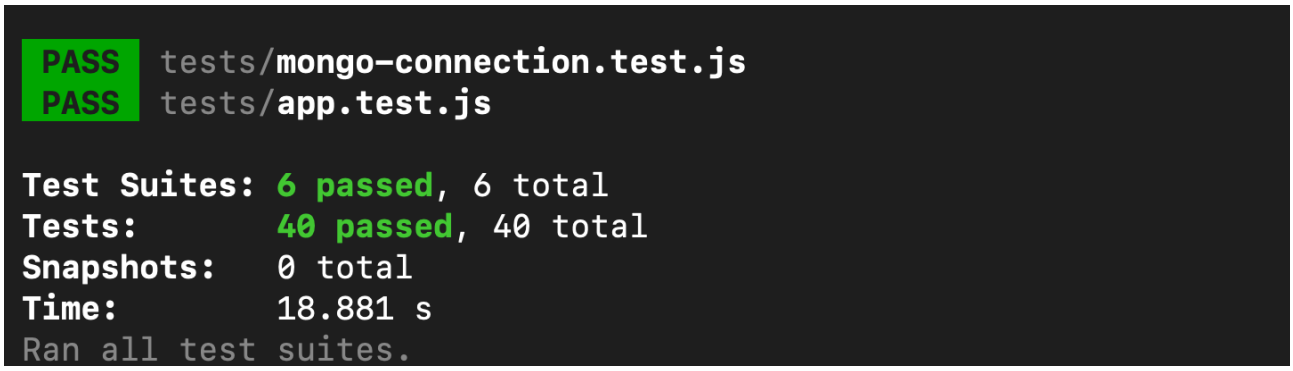
Test files executed:

Sl. No.	Test file	Description
1	admin.test.js	Tests administrative functionality
2	complaint.test.js	Tests complaint-related operations
3	auth.test.js	Tests authentication functionality
4	authMiddleware.test.js	Tests authentication middleware
5	mongo-connection.test.js	Tests MongoDB connection handling
6	app.test.js	Tests Express application functionality

4.1 Backend Test Execution Results

Parameter	Result
Total test suites	6
Passed test suites	6
Failed test suites	0
Total test cases	40
Passed test cases	40

Parameter	Result
Failed test cases	0
Execution time	18.881 seconds



Jest output: 6 of 6 test suites and 40 of 40 tests passed (18.881 s).

Backend testing successful

All six backend test suites passed successfully, with 40 out of 40 test cases passing.

5. Frontend Unit Testing

The frontend is a React application built with Vite. Frontend unit testing verifies the behavior of its components, including the application shell and the dashboard views.

Frontend tests are located in the `client/src/test/` directory.

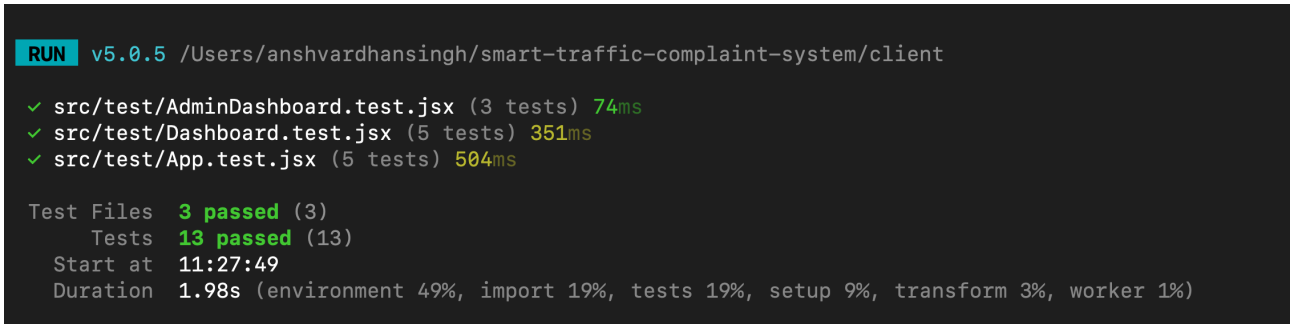
Test files executed:

Sl. No.	Test file	Description	Tests	Time
1	AdminDashboard.test.jsx	Tests the administrator dashboard component	3	74 ms
2	Dashboard.test.jsx	Tests the dashboard component	5	351 ms
3	App.test.jsx	Tests the main application component	5	504 ms

5.1 Frontend Test Execution Results

Parameter	Result
Total test files	3
Passed test files	3
Failed test files	0
Total test cases	13
Passed test cases	13
Failed test cases	0

Parameter	Result
Execution time	1.98 seconds



Vitest output: 3 of 3 test files and 13 of 13 tests passed (1.98 s).

Frontend testing successful

All three frontend test files passed successfully, with 13 out of 13 test cases passing.

6. Deep-Learning Module Unit Testing

The deep-learning module is responsible for analyzing uploaded road images and classifying them into three categories:

1. Road Damage
2. Normal Road
3. Pothole

The system uses a trained ResNet18 model implemented using PyTorch.

The FastAPI application provides endpoints for checking model availability and performing image predictions.

Main source files:

File	Description
src/api.py	Implements the FastAPI prediction API
src/predict.py	Implements image-based prediction functionality
models/best_model.pth	Stores trained model weights

6.1 Deep-Learning Test Cases

The DL module was tested using Python's unittest framework and FastAPI TestClient.

Test execution command:

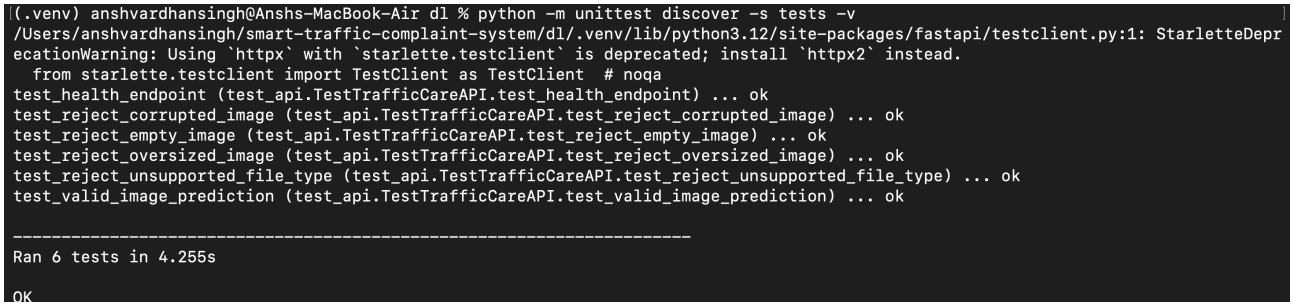
```
python -m unittest discover -s tests -v
```

The following test cases were executed:

Test ID	Test case	Expected result	Status
DL-01	Health endpoint	Returns successful response and model-loaded status	PASS
DL-02	Unsupported file type	Rejects unsupported file formats	PASS
DL-03	Empty image upload	Rejects empty image files	PASS
DL-04	Oversized image upload	Rejects images exceeding 10 MB	PASS
DL-05	Corrupted image upload	Rejects invalid or corrupted images	PASS
DL-06	Valid image prediction	Returns prediction category, confidence and probabilities	PASS

6.2 Deep-Learning Test Execution Results

Parameter	Result
Total test cases	6
Passed test cases	6
Failed test cases	0
Execution time	4.255 seconds



unittest output: 6 tests ran in 4.255 s, result OK.

Deep-learning testing successful

All six deep-learning test cases passed successfully.

Note: The valid prediction test used a mock model to generate predictable outputs. It verifies API prediction handling and response structure, not the real model's classification accuracy.

7. Consolidated Unit Testing Results

The following table summarizes the results obtained from backend, frontend and deep-learning unit testing.

Parameter	Backend	Frontend	Deep Learning	Total
Test suites / files	6	3	1	10
Test cases	40	13	6	59
Passed	40	13	6	59
Failed	0	0	0	0

59 / 59

Unit Tests Passed

100% of executed unit tests passed

40	13	6	0
Backend 40 passed	Frontend 13 passed	Deep Learning 6 passed	Failed tests 0

8. Warnings and Observations

During test execution, certain warnings and log messages were observed.

8.1 Backend observations

- Mongoose displayed a deprecation warning regarding the new option used in findOneAndUpdate() and findOneAndReplace().
- Node.js displayed an experimental VM Modules warning.
- The authentication middleware logged a message when JWT_SECRET was not configured. This occurred during a test that exercised the missing-secret scenario.

8.2 Deep-learning observations

- Starlette displayed a deprecation warning concerning the use of httpx with its TestClient.
- The warning did not affect test execution.

These warnings did not cause any unit test failures. They can be addressed in future maintenance.

9. Limitations of Testing

1. The deep-learning prediction test used a mock model instead of evaluating actual classification accuracy.
2. Unit testing does not independently establish the accuracy of the trained ResNet18 model.
3. Performance testing under heavy concurrent user traffic was not part of this unit-testing execution.

4. Browser-level end-to-end testing was not part of this unit-testing execution.
5. Additional edge cases and integration scenarios can be included in future testing.

10. Conclusion

Unit testing was successfully performed on the TrafficCare Smart Traffic Complaint System.

The backend was tested using Jest across six test suites, covering authentication, complaint management, administrative functionality, middleware and database connectivity.

The frontend was tested using Vitest across three test files containing 13 test cases, covering the application and dashboard components.

The deep-learning API was tested using Python's unittest framework and FastAPI TestClient. The tests verified health-check functionality, image validation, file-size restrictions, corrupted image handling and prediction response generation.

A total of 59 unit test cases were executed, and all 59 passed successfully, with zero failures.

The results demonstrate that the tested backend, frontend and deep-learning API components behave as expected for the tested scenarios.

Unit testing has therefore helped improve the reliability and correctness of the TrafficCare application.